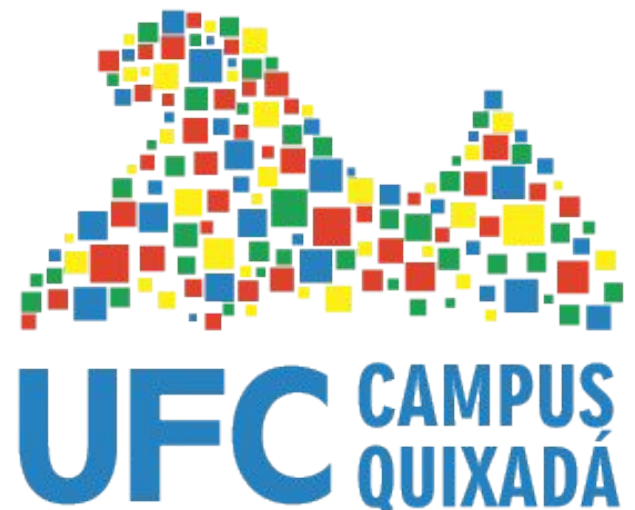


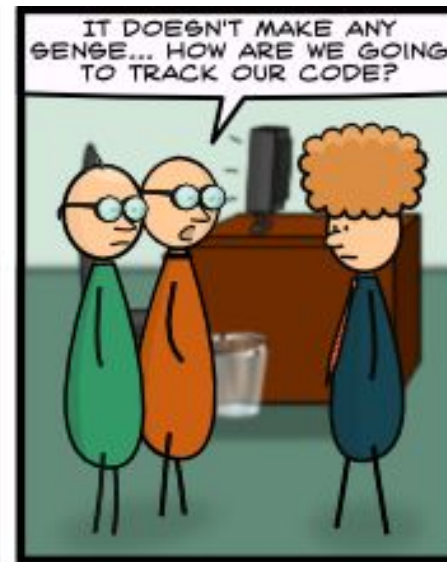
Gerência de Configuração

GC

Sidarta Carvalho



Sistema Git



#101 - "WHO NEEDS GIT?" - BY SALVATORE IOVENE, JAN. 26TH 2009

[HTTP://WWW.GEEKHEROCOMIC.COM/](http://www.geekherocomic.com/)

<http://www.geekherocomic.com/2009/01/26/who-needs-git/>

GitHub



GitLab



Bitbucket



Bitbucket

Sistema Git

A Vida Sem Controle de Versão

- A "Síndrome do Arquivo Final"
 -  projeto_final.zip
 -  projeto_final_v2.zip
 -  projeto_final_AGORA_VAI.zip

Trabalhar em equipe assim gera:

- Medo contínuo
 - Fazer uma grande mudança, quebrar o código e não ter como voltar atrás.

Caos colaborativo

Alguém sobrescrever o seu trabalho na hora de juntar os arquivos por pendrive ou nuvem.

Sistema Git

- Git
 - Histórico, segurança e colaboração — de graça
- Histórico completo
 - Cada mudança fica registrada.
 - Voltar no tempo é um comando, não um milagre.
- Branches
 - Teste, experimente e quebre coisas sem medo, o código principal continua intacto.
- Colaboração real
 - Vários devs trabalhando ao mesmo tempo, sem pisar no trabalho um do outro.
- Backup natural
 - seu código nunca depende só do seu computador.

Git não é burocracia — é a rede de segurança que te dá liberdade para criar.

Sistema Git

- Todo projeto sério usa Git.
- Empresas, open source, projetos acadêmicos — Git é o padrão da indústria
- Aprender Git agora é investir em uma habilidade que você vai usar a vida toda
- Cada commit é um passo documentado da sua evolução como desenvolvedor

Não é sobre nunca errar. É sobre nunca perder o que você construiu. (Gemini, 2026)

- **Git**

- Sistema de controle de versão distribuído, criado para rastrear e gerenciar mudanças em arquivos de código ao longo do tempo.
- Ele permite que desenvolvedores salvem "snapshots" do projeto a cada alteração (commits)
- Voltem a versões anteriores quando necessário
- Criem ramificações (branches) para testar novas funcionalidades sem afetar o código principal

- **Git**

- Colaborem com outras pessoas no mesmo projeto sem sobrescrever o trabalho umas das outras.
- Por ser distribuído, cada desenvolvedor tem uma cópia completa do histórico do projeto em sua própria máquina, o que garante segurança, flexibilidade para trabalhar offline e facilidade de integração com plataformas como GitHub, GitLab e Bitbucket.

Sistema Git



- **Working Directory**
 - É a pasta do seu projeto no computador, onde você efetivamente edita os arquivos.
 - Mudanças aqui ainda não são "vistas" pelo Git como prontas para registro.
- **Staging Area / Index**
 - Área intermediária onde você seleciona exatamente quais mudanças farão parte do próximo commit.
 - O comando [`git add`] move os arquivos modificados da Working Directory até aqui.
- **Repository (.git)**
 - Histórico permanente do projeto, guardado na pasta oculta `.git`.
 - O comando `git commit` grava tudo que está na Staging Area como um novo ponto no histórico, com data, autor e mensagem.

Sistema Git

- Download e instalar a partir do site oficial
 - Windows, Mac ou Linux
- Criar uma pasta e definir a pasta que será o repositório
 - usar o comando: `git init`
- Resgatar repositório de um outro local remoto
 - `git clone http://www.meusite.com/projeto`

Sistema Git

- O [`git config`] é usado para definir configurações do Git em três níveis diferentes:

Nível	Flag	Onde fica salvo	Escopo
System	<code>--system</code>	Configuração da máquina toda	Todos os usuários do computador
Global	<code>--global</code>	<code>~/.gitconfig</code>	Todos os repositórios do seu usuário
Local	(padrão, sem flag)	<code>.git/config</code>	Apenas o repositório atual

```
git config --global user.name "Maria Silva"
git config --global user.email "maria@email.com"
```

Dados que serão usados para registrar seus commits.
A autenticação é feita na plataforma do Git, pode usar qualquer nome ou email no git config.
O que colocar no git config vai aparecer nos commits da plataforma

Sistema Git: branches

- Branch (ramificação) é uma linha independente de desenvolvimento.
- Do ponto de vista técnico, o Git gerencia as branches de forma extremamente leve
- branch é apenas um ponteiro móvel que aponta para um commit específico no histórico do repositório.
- Quando você cria uma nova branch, o projeto é "clonado" instantaneamente naquele ponto do tempo, permitindo que você altere, adicione ou remova arquivos sem afetar a linha principal do projeto (geralmente chamada de main).

Sistema Git: branches

- A utilização de branches resolve o maior desafio de equipes de software
 - Trabalho assíncrono e paralelo.
 - Sem elas, os desenvolvedores estariam constantemente sobrescrevendo o trabalho uns dos outros ou paralisando o projeto inteiro caso uma nova funcionalidade quebrasse o sistema.

Sistema Git: branches

1. Desenvolvimento de Novas Funcionalidades
2. Uma equipe trabalhando em um e-commerce precisa integrar uma nova API de pagamentos (ex: PIX), o que levará duas semanas.
3. O desenvolvedor cria uma branch chamada feat/pagamento-pix a partir da main.
4. Ele trabalha ali diariamente, fazendo commits isolados.
5. Durante essas duas semanas, a branch main continua estável e funcional.
6. Quando o código do PIX está pronto e testado, ele é integrado (merged) à main via um Pull Request (merge request).

Sistema Git: branches

1. Em plena sexta-feira de Black Friday, descobre-se que o botão de "Comprar" parou de funcionar na produção.
2. Ao mesmo tempo, existem desenvolvedores trabalhando no novo layout do site em suas respectivas feature branches.
3. A equipe não pode esperar o novo layout ficar pronto.
4. Alguém cria uma branch [hotfix/botao-comprar] diretamente a partir da versão que está em produção.
5. O bug é corrigido, testado e fundido imediatamente para a produção, sem se misturar com o código incompleto das outras funcionalidades.

Sistema Git: branches

1. O software está prestes a mudar para a versão 2.0.
2. Cria-se uma branch release/v2.0.
3. A partir desse momento, nenhuma nova funcionalidade entra nessa branch, apenas correções de bugs de última hora e ajustes de documentação.
4. Enquanto isso, o resto da equipe já pode começar a trabalhar nas funcionalidades da versão 2.1 na main.

Sistema Git: branches

- **Padrão de Nomenclatura (Naming Conventions)**
 - Em equipes, não se cria branches com nomes como meu-teste ou arrumando-bug.
 - Adota-se um padrão que geralmente inclui o tipo da tarefa e o ID do ticket.
- **feat/auth-google** (Nova funcionalidade de autenticação)
- **bugfix/JIRA-123-erro-calculo** (Correção de um bug rastreado)
- **docs/atualiza-readme** (Apenas mudanças de documentação)

Sistema Git: caminho felix



Sistema Git: principais comandos

Inicializar Repositório

```
git init
```

Navegação & Checkout

```
git checkout main
```

Muda o foco para a branch "main".

```
git checkout -b feature-login
```

Cria uma nova branch e já muda para ela.

```
git checkout -- arquivo.txt
```

Descarta alterações não salvas no arquivo.

```
git switch -c nome-nova-branch
```

Criar uma nova branch e muda para ela automaticamente

Dica Moderna:

O comando `git switch` é recomendado para alternar entre branches.

Gerenciar Branches

```
git branch
```

Lista todas as branches locais existentes.

```
git branch nova-feature
```

Cria uma nova branch (sem mudar para ela).

```
git branch -d nova-feature
```

Exclui de forma segura (apenas se já mesclada).

```
git branch -D nova-feature
```

Força a exclusão da branch (mesmo sem merge).

Sistema Git: principais comandos

Começando

`git init`

Cria um novo repositório Git local na pasta atual.

`git status`

Lista os arquivos modificados e os que estão prontos para o envio.

Preparar (git add)

Adiciona as alterações ao Stage (preparação):

`git add arquivo1.txt`

Adiciona um arquivo específico.

`git add *.txt`

Adiciona todos os arquivos de texto (.txt).

`git add .`

Ponto final: Adiciona todas as modificações do diretório.

Gravar Versão

`git commit -m "mensagem"`

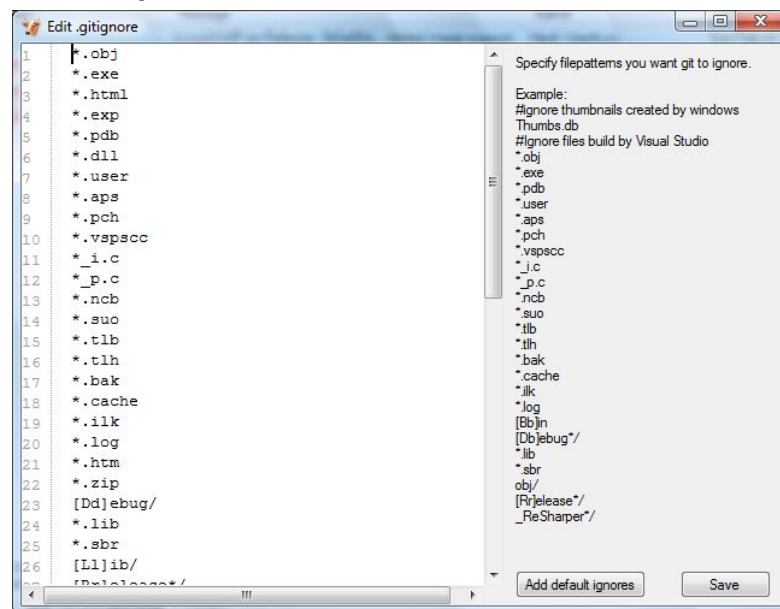
Cria um ponto na história salvando as alterações preparadas.

Nota importante:

Não pergunta arquivo por arquivo. O commit engloba tudo o que foi adicionado no git add de uma só vez.

Sistema Git

- Arquivo “.gitignore”
 - Armazena quais arquivos devem ser ignorados pelo git no versionamento.
 - Exemplo: x.txt e y.txt estão no .gitignore, logo, modificações nestes arquivos não serão consideradas como parte da versão/working directory.



Sistema Git: comandos

Comparar Local

`git diff`

Identifica mudanças entre as versões que já receberam commit e o seu estado atual no **working directory**.

- Exibe detalhadamente todas as diferenças nas versões.

Atenção: Após utilizar o `git add`, as modificações deixam de ser rastreadas por este comando.

Comparar com Último Commit

`git diff HEAD`

Mostra as diferenças exatas entre o seu ambiente de desenvolvimento atual e o estado do último commit que foi realizado.

Útil para revisar tudo o que foi alterado localmente antes de fechar uma nova versão ou fazer um novo commit.

Sistema Git: comandos

- `git log`
 - Histórico dos commits. Há uma chave gerada automaticamente para cada commit que permite voltar a uma determinada versão do sistema
- `git log -p`
 - Fornece mais detalhes, trazendo o histórico com as diferenças (DIFFs) de cada arquivo
 - Letra “q” é usada para sair da paginação ;-)
- `git log -p -1`
 - Lista somente 1 commit, o último!
- Dica: é possível utilizar uma interface gráfica para visualizar melhor o log.

Sistema Git: comandos

- **git tag**
 - Marca um commit específico como um ponto importante (ex: versão de release).
- `git tag v1.0`
 - cria uma tag simples
- `git tag -a v1.0 -m "Primeira versão"`
 - tag anotada (com mensagem)
- `git tag`
 - lista todas as tags

Sistema Git: comandos

- Errei e acabei fazendo um commit antes da hora... e agora?
- `git log --pretty=oneline`
 - Lista a chave da versão e a mensagem do commit
 - Lembra do commit `-m` “mensagem”?
- `git commit --amend “texto mensagem”`
 - Editar um commit
 - Substitui o commit gravado pela working área atual

Sistema Git: comandos

- **Adicionei um arquivo que não deveria... e agora?**
- `git reset HEAD arquivoErrado.txt`
 - Desfaz um `git add`.
 - Ou seja, pega um arquivo que estava pronto para ser commitado (na Staging Area) e o "rebaixa" de volta para o seu Working Directory.
- Na prática: Imagine que você editou 3 arquivos, mas só queria salvar 2 no próximo commit. Você, por acidente, digitou `git add .` (adicionando os 3).
- Para tirar o arquivo indesejado da fila do commit sem perder as alterações que você digitou nele, você usa o `git reset HEAD`. O seu código permanece intacto no computador, apenas não vai mais entrar no próximo pacote.

Sistema Git: comandos

- **Adicionei um arquivo que não deveria... e agora?**
 - `git checkout --nomeArquivo.txt`
 - Pega a versão do arquivo que está salva no último commit e joga por cima do seu arquivo atual no Working Directory.
 - Na prática: Você abriu um arquivo, começou a apagar e reescrever código e, de repente, quebrou o sistema todo.
 - Você se arrepende e quer que o arquivo volte a ser exatamente como era no último commit. Esse comando faz exatamente isso, como um "desfazer" absoluto.
 - Este é um comando destrutivo. Se você rodar isso, todas as alterações que não haviam sido salvas (commitadas) naquele arquivo serão perdidas para sempre e não há como recuperar.

Sistema Git: comandos

- Hoje em dia, a recomendação oficial do Git é substituir esses comandos clássicos por estes novos:
 - Em vez de [`git reset HEAD arquivo.txt`]
 - Use [`git restore --staged arquivo.txt`]
 - Tira da Staging Area de forma muito mais semântica.
 - Em vez de [`git checkout -- arquivo.txt`]
 - Use [`git restore arquivo.txt`]
 - Descarta as mudanças e restaura o arquivo no Working Directory.

Sistema Git: comandos

- **git push -u origin <nome-da-branch>**
 - Quando você cria uma branch localmente, ela não existe no servidor.
 - Esse comando faz o upload da sua branch para o servidor (origin) e o -u (set-upstream) cria um elo entre a sua branch local e a remota.
 - Nas próximas vezes, bastará digitar git push.

Sistema Git: comandos

- **git fetch**
 - Baixa as atualizações do servidor remoto (incluindo branches que seus colegas criaram), mas não as aplica automaticamente no seu código.
- **git branch -r**
 - Lista as branches remotas (aquelas que estão no servidor, mas que você pode ainda não ter na sua máquina).

Sistema Git: comandos

Para o Git local conseguir enviar (push) código para o seu GitHub, você precisa provar quem você é. Formas de autenticação:

- 1. GitHub CLI / Navegador (Mais fácil para iniciantes):
 - 1. Baixar e [`gh auth login`]
 - O terminal abre uma janela do navegador para você logar.
- 2. Personal Access Token (PAT):
 - Uma senha especial gerada nas configurações do GitHub, usada em conexões HTTPS.
- 3. Chaves SSH (RSA/Ed25519) (Mais seguro e profissional):
 - Geração de chaves assimétricas. A chave pública fica no GitHub e a privada no seu computador, eliminando a necessidade de digitar senhas a cada push.

Servidores GIT na nuvem

- Empresas utilizam o github (e similares) para buscar projetos parecidos com o que desejam construir.
- Melhor forma de analisar um candidato de forma técnica
 - Ver o que ele já fez
 - Foco em resolver o problema
- Mantenham estes projetos simples em seus repositórios e usem o link em seus currículos

- Agora já sei tudo de Git, posso começar a contribuir com o kernel Linux?
 - Vai com calma pequeno gafanhoto rssrss
 - Contribuir com projetos grandes exige experiência técnica e conhecimento aprofundado nas técnicas de versionamento do código
- Ao aumentar seu portfólio, não esquecer de documentar bem o que o seu projeto faz
 - Utilidade do projeto
 - Descrever de forma simples para não técnicos
 - Usar boas práticas de escrita de código

- Nos projetos do Github há um arquivo chamado README.MD
 - Este arquivo padrão para os projetos contém a visualização da apresentação do projeto.
 - É um código HTML
 - Imagens, tags, colorações, etc

Dúvidas?



www.shutterstock.com · 744867163